LAPORAN PRAKTIKUM

Inheritance, Generalization & Overriding

Dibuat untuk memenuhi salah satu tugas Praktikum Mata Kuliah
Pengembangan Berorientasi Objek



Dibuat Oleh:

NAMA	: Maulida Nur Afifah
NIM	: 251511017
KELAS	: 2-A
PROGRAM STUDI	: D3 Teknik Informatika
JURUSAN	: Teknik Komputer dan Informatika

POLITEKNIK NEGERI BANDUNG

2026

PRAKTIKUM I
CIRCLE AND CYLINDER

Circle.java

```
public class Circle {
    // private instance variable
    private double radius;
    private String color;

    // constructs a circle instance with the given radius default color
    public Circle() {
        radius = 1.0;
        color = "red";
    }
    // constructs a circle instance with the given radius and default color
    public Circle(double radius) {
        this.radius = radius;
        color = "red";
    }
    // return the radius
    public double getRadius() {
        return radius;
    }
    // return the area of this circle instance
    public double getArea() {
        return radius * radius * Math.PI;
    }
    // return a self-descriptive string of this instance in the form of circle
    public String toString() {
        return "Circle[radius=" + radius + ", color=" + color + "]";
    }

    public Circle(double radius, String color){
        this.radius = radius;
        this.color = color;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }

    public String getColor() {
        return color;
    }

    public void setColor(String color) {
        this.color = color;
    }
}
```

Checkpoint 1

State Circle: radius (double), color (string)
Behavior Circle: getRadius(). getArea(), toString()
Perbedaan state dua object:

	Circle()	Circle(2.5)
radius	1.0 (default)	2.5 (dari parameter)
color	“red”	“red” (hardcoded di constructor)

Checkpoint 2

Code di luar Circle tidak bisa mengubah radius langsung karena field itu private.
Method public (getter dan setter) berperan sebagai pintu masuk ke state private yang dikendalikan oleh class pemiliknya apakah statenya boleh dibaca atau diubah

Cylinder.java

```
public class Cylinder extends Circle{
    private double height;

    public Cylinder() {
        super();
        height = 1.0;
    }

    public Cylinder(double height) {
        super();
        this.height = height;
    }

    public Cylinder(double radius, double height) {
        super(radius);
        this.height = height;
    }

    public double getHeight() {
        return height;
    }

    public void setHeight(double height) {
        this.height = height;
    }

    public double getVolume() {
        return getArea() * height;
    }
}
```

TestCylinder.java

```
public class TestCylinder {
    public static void main(String[] args) {
        Cylinder c1 = new Cylinder();
        Cylinder c2 = new Cylinder(10.0);
        Cylinder c3 = new Cylinder(2.0, 10.0);

        printCylinder("c1", c1);
        printCylinder("c2", c2);
        printCylinder("c3", c3);
    }

    public static void printCylinder(String name, Cylinder c){
        System.out.println(name);
        System.out.println("radius    = " + c.getRadius());
        System.out.println("height    = "+ c.getHeight());
        System.out.println("base area = "+ c.getArea());
        System.out.println("volume    = "+ c.getVolume());
        System.out.println();
    }
}
```

Output TextCylinder

```
D:\POLBAN-2025\smt3\PBO\Praktek\praktikum4\source>java TestCylinder
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encodi
ng=UTF-8
c1
radius    = 1.0
height    = 1.0
base area = 3.141592653589793
volume    = 3.141592653589793

c2
radius    = 1.0
height    = 10.0
base area = 3.141592653589793
volume    = 31.41592653589793

c3
radius    = 2.0
height    = 10.0
base area = 12.566370614359172
volume    = 125.66370614359172
```



Checkpoint 3

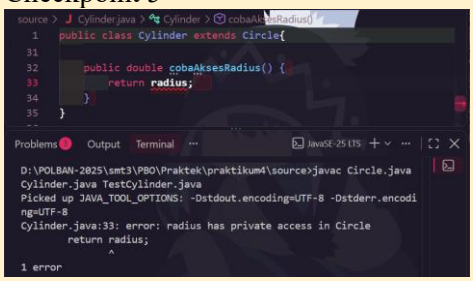
c3.getRadius() dan c3.getArea() bisa dipanggil karena Cylinder extends Circle. extends membuat Cylinder mewarisi seluruh behavior public milik Circle tanpa perlu ditulis ulang. Kedua method itu milik Circle, tapi object Cylinder otomatis membawanya

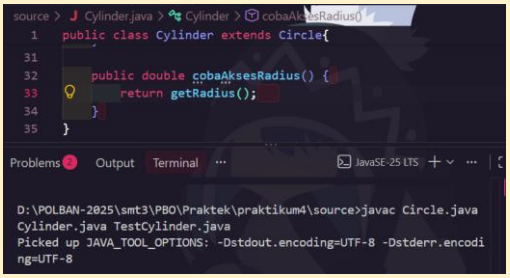
Checkpoint 4

```
Circle(double) dipanggil
Cylinder(double,double) dipanggil
```

Circle dikerjakan lebih dulu. Buktinya: output Circle muncul sebelum output Cylinder. Ini terjadi karena baris pertama di constructor Cylinder adalah super(radius). Java mengharuskan bagian superclass selesai diinisialisasi sebelum bisa melanjutkan subclass.
Kalau super(radius) dihapus akan error

Checkpoint 5





extends membuat subclass mewarisi behavior yang dapat diakses, tetapi tidak membuat field private superclass menjadi bisa diakses langsung oleh subclass

Checkpoint 6

Setelah getArea() dioverride di Cylinder menjadi luas permukaan silinder, method getVolume() menjadi salah karena di dalamnya ada getArea() * height.
Akibatnya: getVolume() menghitung luas permukaan × height, bukan luas alas × height.

Checkpoint 7

Method tetap bisa meng-override tanpa @Override. Kegunaannya sebagai pemeriksaan compiler, compiler akan coba memverifikasi apakah ada methid dengan nama ini di superclass atau tidak. Jika tidak ada, compiler langsung error. Tanpa @Override, method yang salah nama hanya jadi method baru yang berbeda.

PRAKTIKUM II
SHAPE HIERARCY

Buat superclass Shape

Shape.java

```
public class Shape {
     private String color;
    private boolean filled;

    public Shape() {
        color = "green";
        filled = true;
    }

    public Shape(String color, boolean filled) {
        this.color = color;
        this.filled = filled;
    }

    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }
    public boolean isFilled() { return filled; }
    public void setFilled(boolean filled) { this.filled = filled; }

    @Override
    public String toString() {
        return "Shape[color=" + color + ", filled=" + filled + "]";
    }
}
```

Checkpoint 8

State yang cukup umum untuk dimiliki semua bentuk: color dan filler (status terisi/tidak)

Buat circle sebagai specialization dari Shape

Circle.java

```
public class Circle extends Shape {
    // private instance variable
    private double radius;

    public Circle() {
        super();
        radius = 1.0;
    }

    public Circle(double radius) {
        super();
        this.radius = radius;
    }

     public Circle(double radius, String color, boolean filled) {
        super(color, filled);
        this.radius = radius;
    }

    public double getRadius() { return radius; }
    public void setRadius(double radius) { this.radius = radius; }

    public double getArea() { return Math.PI * radius * radius; }
    public double getPerimeter() { return 2 * Math.PI * radius; }
```

```
    @Override
    public String toString() {
        return "Circle[radius=" + radius + ", " + super.toString() + "]";
    }
}
```

Checkpoint 9

Dari Shape (diwarisi)	Spesifik milik Circle
getColor()	Radius
setColor()	getRadius()
isFiller()	setRadius()
setFilled()	getArea()
Bagian color+filler di toString()	getPerimeter()
	Bagian radius di toString()

Buat rectangle sebagai specialization dari Shape

Rectangle.java

```
public class Rectangle extends Shape{
     private double width;
    private double length;

    public Rectangle() {
        super();
        width = 1.0;
        length = 1.0;
    }

    public Rectangle(double width, double length) {
        super();
        this.width = width;
        this.length = length;
    }

    public Rectangle(double width, double length, String color, boolean filled) {
        super(color, filled);
        this.width = width;
        this.length = length;
    }

    public double getWidth() { return width; }
    public void setWidth(double width) { this.width = width; }
    public double getLength() { return length; }
    public void setLength(double length) { this.length = length; }

    public double getArea() { return width * length; }
    public double getPerimeter() { return 2 * (width + length); }

    @Override
    public String toString() {
        return "Rectangle[width=" + width + ", length=" + length
               + ", " + super.toString() + "]";
    }
}
```

Checkpoint 10

	Circle	Rectangle
Dimensi	Radius	Width, length
getArea()	Pi*r^2	Width * length

getPerimeter()	2*pi*r	2 * (width + length)
toString()	Menampilkan radius	Menampilkan width dan length

Buat Square sebagai dari Rectangle

Square.java

```
public class Square extends Rectangle{
    public Square() {
        super(1.0, 1.0);
    }

    public Square(double side) {
        super(side, side);
    }

    public Square(double side, String color, boolean filled) {
        super(side, side, color, filled);
    }

    public double getSide() {
        return getWidth();
    }

    public void setSide(double side) {
        super.setWidth(side);
        super.setLength(side);
    }

    @Override
    public String toString() {
        return "Square[side=" + getSide() + ", " + super.toString() + "]";
    }
}
```

Checkpoint 11

Object tersebut tidak lagi merepresentasikan Square karena syarat utama Square adalah width selalu sama dengan length

Checkpoint 12

```
Sesudah setLength(3) : Square[side=3.0, Rectangle[width=3.0, length=3.0, Shape[color=green, filled=true]]]
Area     : 9.0
Perimeter: 12.0

getArea() dan getPerimeter() tidak dioverride di Square.
Hasilnya tetap benar karena setter sudah menjaga width == length.
```

setWidth() dan setLength() perlu dioverride karena mengubah salah satu dimensi saja melanggar kontrak Square. Setiap dimensi berubah, dua-duanya harus berubah juga dan override memastikan aturan ini selalu terpenuhi.
getArea() dan getPerimeter() tidak perlu dioverride karena formula untuk area dan perimeter tetap menghasilkan nilai benar untuk square.

BAGIAN C
KASUS INVENTARIS

Kandidat Generalization

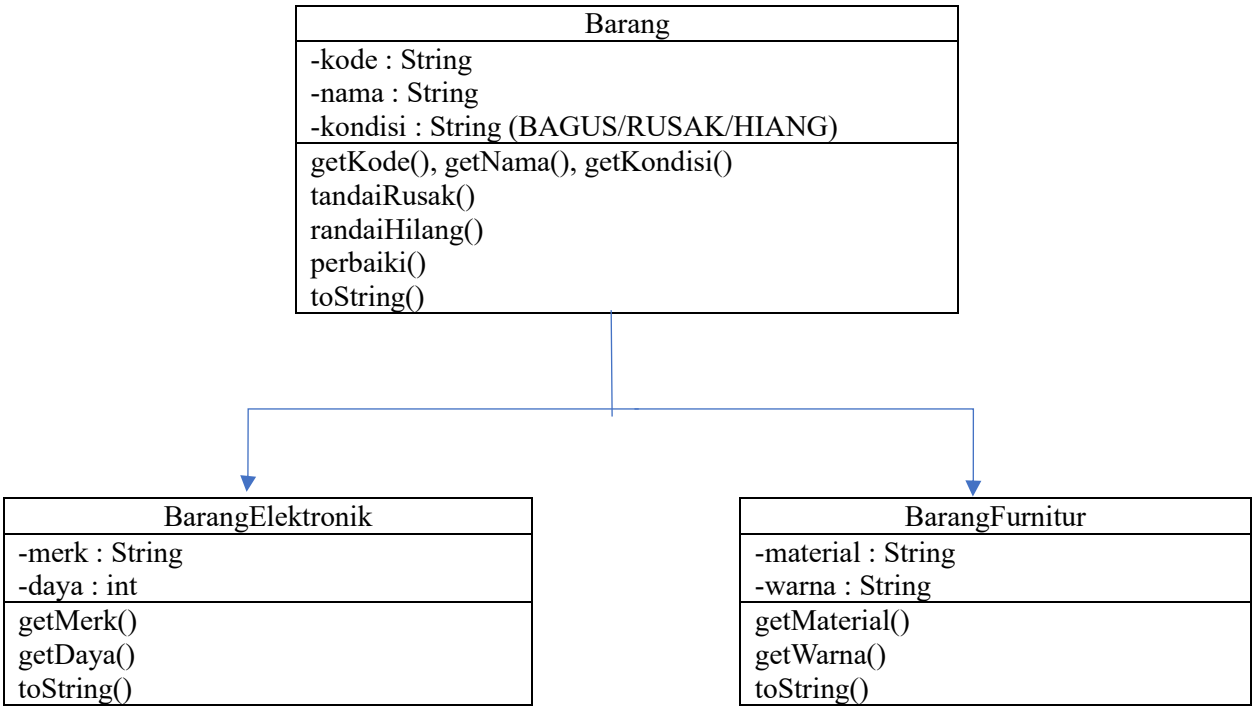
Dua class dengan state/behavior mirip yang bisa dijadikan subsclass:

	State/Behavior yang sama	State/Behavior yang beda
BarangElektronik	Kode, nama, kondisi, tandaiRusak(), tandaiHilang(), perbaiki()	Merk, daya
BarangFurnitur	Kode, nama, kondisi, tandaiRusak(), tandaiHilang(), perbaiki()	Material, warna

BarangElektronik is a Barang

BarangFurnitur is a Barang

Diagram Hierarchy



Barang.java

```
public class Barang {

    public static final String BAGUS  = "BAGUS";
    public static final String RUSAK  = "RUSAK";
    public static final String HILANG = "HILANG";

    private String kode;
    private String nama;
    private String kondisi;

    public Barang(String kode, String nama) {
```

```
        this.kode    = kode;
        this.nama    = nama;
        this.kondisi = BAGUS;
    }

    public String getKode() { return kode; }
    public String getNama() { return nama; }
    public String getKondisi() { return kondisi; }

    public void tandaiRusak() {
        this.kondisi = RUSAK;
    }

    public void tandaiHilang() {
        this.kondisi = HILANG;
    }

    public void perbaiki() {
        this.kondisi = BAGUS;
    }

    @Override
    public String toString() {
        return "kode=" + kode
             + ", nama=" + nama
             + ", kondisi=" + kondisi;
    }
}
```

BarangElektronik.java

```
public class BarangElektronik extends Barang {
    private String merek;
    private int daya;

    public BarangElektronik(String kode, String nama, String merek, int daya) {
        super(kode, nama);
        this.merek = merek;
        this.daya  = daya;
    }

    public String getMerek() { return merek; }
    public int getDaya() { return daya;  }

    @Override
    public String toString() {
        return "BarangElektronik["
             + super.toString()
             + ", merek=" + merek
             + ", daya="  + daya + "W]";
    }
}
```

BarangFurnitur.java

```
public class BarangFurnitur extends Barang {
    private String material;
    private String warna;

    public BarangFurnitur(String kode, String nama, String material, String warna) {
        super(kode, nama);
        this.material = material;
        this.warna = warna;
    }

    public String getMaterial() { return material; }
    public String getWarna() { return warna;    }
```

```
    @Override
    public String toString() {
        return "BarangFurnitur["
             + super.toString()
             + ", material=" + material
             + ", warna="    + warna + "]";
    }
}
```

Lokasi.java

```
public class Lokasi {
    private String kodeLokasi;
    private String namaLokasi;

    public Lokasi(String kodeLokasi, String namaLokasi) {
        this.kodeLokasi = kodeLokasi;
        this.namaLokasi = namaLokasi;
    }

    public String getKodeLokasi() {return kodeLokasi;}
    public String getNamaLokasi() {return namaLokasi;}

    public String getLokasi() {
        return "=== Data Lokasi ===\n"
        + "Kode        : " + kodeLokasi + "\n"
        + "Nama Lokasi : " + namaLokasi + "\n";
    }
}
```

PencatatanInventaris.java

```
public class PencatatanInventaris {
    private String kode;
    private Barang barang;
    private Lokasi lokasi;

    public PencatatanInventaris(String kode, Barang barang, Lokasi lokasi) {
        this.kode = kode;
        this.barang = barang;
        this.lokasi = lokasi;
    }

    public String getKode() {return kode;}
    public Barang getBarang() {return barang;}
    public Lokasi getLokasi() {return lokasi;}

    public void tampilInfo() {
        System.out.println("=== Hasil Pencatatan ===");
        System.out.println("Kode        : "+ kode);
        System.out.println("Nama Barang : "+ barang.getNama());
        System.out.println("Kondisi     : "+ barang.getKondisi());
        System.out.println("Lokasi      : "+ lokasi.getNamaLokasi());
        System.out.println("Detail Barang   : " + barang);
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {

        // ============================================================
        // BASE CASE Tugas Kelas: Inventaris Barang Kampus
        // ============================================================

        // 2 Lokasi
        Lokasi labKomputer = new Lokasi("L01", "Lab Komputer");
```

```
        Lokasi ruangKelas  = new Lokasi("L02", "Ruang Kelas 3A");

        // 2 Barang (tipe dasar, sebelum inheritance)
        Barang proyektor = new Barang("B01", "Proyektor Epson");
        Barang kursi     = new Barang("B02", "Kursi Kuliah");

        // 2 PencatatanInventaris
        PencatatanInventaris catatan1 = new PencatatanInventaris("INV-001", proyektor, labKomputer);
        PencatatanInventaris catatan2 = new PencatatanInventaris("INV-002", kursi, ruangKelas);

        System.out.println("=== DATA HASIL PENCATATAN INVENTARIS ===");
        System.out.println("--- Catatan 1 ---");
        catatan1.tampilInfo();
        System.out.println();
        System.out.println("--- Catatan 2 ---");
        catatan2.tampilInfo();
        System.out.println();

        // Tampilkan status sebelum dan sesudah behavior dijalankan
        System.out.println("=== STATUS BARANG SEBELUM DAN SESUDAH BEHAVIOR ===");
        System.out.println("Proyektor kondisi SEBELUM : " + proyektor.getKondisi());
        proyektor.tandaiRusak();
        System.out.println("Proyektor kondisi SESUDAH tandaiRusak()  : "
                + proyektor.getKondisi());
        proyektor.perbaiki();
        System.out.println("Proyektor kondisi SESUDAH perbaiki()     : "
                + proyektor.getKondisi());
        System.out.println();

        System.out.println("Kursi kondisi SEBELUM : " + kursi.getKondisi());
        kursi.tandaiHilang();
        System.out.println("Kursi kondisi SESUDAH tandaiHilang() : "
                + kursi.getKondisi());
        System.out.println();

        // ============================================================
        // MODUL 4 — INHERITANCE: Barang → BarangElektronik / BarangFurnitur
        // ============================================================

        // T4-01 — Object subclass dibuat melalui constructor
        // super() di setiap constructor menginisialisasi bagian Barang
        System.out.println("=== T4-01: Constructor Subclass + super() ===");

        BarangElektronik laptop   = new BarangElektronik(
                                        "E01", "Laptop Lenovo", "Lenovo", 65);
        BarangElektronik ac       = new BarangElektronik(
                                        "E02", "AC Ruangan", "Daikin", 900);
        BarangFurnitur   mejaDosen = new BarangFurnitur(
                                        "F01", "Meja Dosen", "Kayu Jati", "Coklat");
        BarangFurnitur   lemari   = new BarangFurnitur(
                                        "F02", "Lemari Arsip", "Besi", "Abu-abu");

        // Bukti bagian Barang terinisialisasi melalui super():
        // getNama(), getKode(), getKondisi() berasal dari Barang
        System.out.println("laptop.getNama()    → " + laptop.getNama());
        System.out.println("laptop.getKondisi() → " + laptop.getKondisi()); // BAGUS (default)
        System.out.println("ac.getKode()        → " + ac.getKode());
        System.out.println("mejaDosen.getNama() → " + mejaDosen.getNama());
        System.out.println();

        // T4-02 — Method warisan dipanggil dari object subclass
        // tandaiRusak(), tandaiHilang(), perbaiki(), getKondisi()
        // tidak ditulis di BarangElektronik/BarangFurnitur, tapi tetap bisa dipanggil
        System.out.println("=== T4-02: Inherited Method dari Subclass ===");
        System.out.println("laptop.getKondisi() sebelum  : " + laptop.getKondisi());
        laptop.tandaiRusak(); // tandaiRusak() diwarisi dari Barang
        System.out.println("laptop.getKondisi() sesudah tandaiRusak()  : "
                + laptop.getKondisi());
        laptop.perbaiki(); // perbaiki() diwarisi dari Barang
```

```
        System.out.println("laptop.getKondisi() sesudah perbaiki()     : "
                + laptop.getKondisi());
        System.out.println();

        System.out.println("lemari.getKondisi() sebelum  : " + lemari.getKondisi());
        lemari.tandaiHilang(); // tandaiHilang() diwarisi dari Barang
        System.out.println("lemari.getKondisi() sesudah tandaiHilang() : "
                + lemari.getKondisi());
        System.out.println();

        // T4-03 – Override toString() dipanggil
        // Java menjalankan versi toString() paling spesifik dari setiap object
        System.out.println("=== T4-03: Override toString() ===");
        System.out.println(laptop);    // → BarangElektronik.toString()
        System.out.println(ac);        // → BarangElektronik.toString()
        System.out.println(mejaDosen); // → BarangFurnitur.toString()
        System.out.println(lemari);    // → BarangFurnitur.toString()
        System.out.println();

        // T4-04 – super.toString() terlihat dalam output T4-03
        // Output BarangElektronik menyertakan kode/nama/kondisi dari Barang.toString()
        // Output BarangFurnitur juga menyertakan bagian yang sama
        System.out.println("=== T4-04: super.toString() dalam Output ===");
        System.out.println("Bukti: output T4-03 mengandung");
        System.out.println("  'kode=..., nama=..., kondisi=...'");
        System.out.println("  → bagian itu berasal dari super.toString() milik Barang");
        System.out.println("  → masing-masing subclass menambahkan atribut spesifiknya");
        System.out.println();

        // T4-05 – Encapsulation tetap terjaga pada hierarchy
        // Field private Barang tidak bisa diakses langsung dari subclass maupun Main
        // Baris berikut jika di-uncomment akan menyebabkan compiler error:
        // laptop.kode    = "X";    → error: kode has private access in Barang
        // laptop.kondisi = "RUSAK";→ error: kondisi has private access in Barang
        System.out.println("=== T4-05: Encapsulation pada Hierarchy ===");
        System.out.println("Field private Barang (kode, nama, kondisi)");
        System.out.println("tidak bisa diakses langsung dari BarangElektronik,");
        System.out.println("BarangFurnitur, maupun dari Main.");
        System.out.println("Subclass hanya bisa akses via getKode(), getNama(), getKondisi().");
        System.out.println();

        // T4-06 – Dua subclass menampilkan output berbeda
        System.out.println("=== T4-06: Dua Subclass, Behavior Berbeda ===");
        System.out.println("BarangElektronik : " + laptop);
        System.out.println("BarangFurnitur   : " + mejaDosen);
        System.out.println("→ toString() berbeda: Elektronik tampilkan merek+daya,");
        System.out.println("  Furnitur tampilkan material+warna");
        System.out.println();

        // T4-07 – Regression: PencatatanInventaris masih bisa pakai subclass
        // field "private Barang barang" di PencatatanInventaris
        // menerima BarangElektronik/BarangFurnitur karena is-a Barang
        System.out.println("=== T4-07: Regression – PencatatanInventaris + Subclass ===");

        Lokasi labElektro = new Lokasi("L03", "Lab Elektronika");
        Lokasi gudang     = new Lokasi("L04", "Gudang Sarana");

        PencatatanInventaris catatan3 =
                new PencatatanInventaris("INV-003", laptop,    labElektro);
        PencatatanInventaris catatan4 =
                new PencatatanInventaris("INV-004", ac,        labElektro);
        PencatatanInventaris catatan5 =
                new PencatatanInventaris("INV-005", mejaDosen, ruangKelas);
        PencatatanInventaris catatan6 =
                new PencatatanInventaris("INV-006", lemari,    gudang);

        System.out.println("--- INV-003 ---");
        catatan3.tampilInfo(); // laptop (BarangElektronik) → tampilInfo() tetap bekerja
        System.out.println();
```

```
        System.out.println("--- INV-004 ---");
        catatan4.tampilInfo(); // ac (BarangElektronik)
        System.out.println();
        System.out.println("--- INV-005 ---");
        catatan5.tampilInfo(); // mejaDosen (BarangFurnitur)
        System.out.println();
        System.out.println("--- INV-006 ---");
        catatan6.tampilInfo(); // lemari (BarangFurnitur, kondisi HILANG)
    }
}
```

Output:

```
[Running] cd "d:\POLBAN-2025\smt3\PBO\Praktek\praktikum4\source\inventaris\" && javac Main.java && java
Main
=== DATA HASIL PENCATATAN INVENTARIS ===
--- Catatan 1 ---
=== Hasil Pencatatan ===
Kode : INV-001
Nama Barang : Proyektor Epson
Kondisi : BAGUS
Lokasi      : Lab Komputer
Detail Barang   : kode=B01, nama=Proyektor Epson, kondisi=BAGUS

--- Catatan 2 ---
=== Hasil Pencatatan ===
Kode : INV-002
Nama Barang : Kursi Kuliah
Kondisi : BAGUS
Lokasi      : Ruang Kelas 3A
Detail Barang   : kode=B02, nama=Kursi Kuliah, kondisi=BAGUS

=== STATUS BARANG SEBELUM DAN SESUDAH BEHAVIOR ===
Proyektor kondisi SEBELUM : BAGUS
Proyektor kondisi SESUDAH tandaiRusak()  : RUSAK
Proyektor kondisi SESUDAH perbaiki()     : BAGUS

Kursi kondisi SEBELUM : BAGUS
Kursi kondisi SESUDAH tandaiHilang() : HILANG

=== T4-01: Constructor Subclass + super() ===
laptop.getNama()    → Laptop Lenovo
laptop.getKondisi() → BAGUS
ac.getKode()        → E02
mejaDosen.getNama() → Meja Dosen

=== T4-02: Inherited Method dari Subclass ===
laptop.getKondisi() sebelum  : BAGUS
laptop.getKondisi() sesudah tandaiRusak()  : RUSAK
laptop.getKondisi() sesudah perbaiki()     : BAGUS

lemari.getKondisi() sebelum  : BAGUS
lemari.getKondisi() sesudah tandaiHilang() : HILANG

=== T4-03: Override toString() ===
BarangElektronik[kode=E01, nama=Laptop Lenovo, kondisi=BAGUS, merek=Lenovo, daya=65W]
BarangElektronik[kode=E02, nama=AC Ruangan, kondisi=BAGUS, merek=Daikin, daya=900W]
BarangFurnitur[kode=F01, nama=Meja Dosen, kondisi=BAGUS, material=Kayu Jati, warna=Coklat]
BarangFurnitur[kode=F02, nama=Lemari Arsip, kondisi=HILANG, material=Besi, warna=Abu-abu]

=== T4-04: super.toString() dalam Output ===
Bukti: output T4-03 mengandung
  'kode=..., nama=..., kondisi=...'
  → bagian itu berasal dari super.toString() milik Barang
  → masing-masing subclass menambahkan atribut spesifiknya

=== T4-05: Encapsulation pada Hierarchy ===
Field private Barang (kode, nama, kondisi)
tidak bisa diakses langsung dari BarangElektronik,
```

```
BarangFurnitur, maupun dari Main.
Subclass hanya bisa akses via getKode(), getNama(), getKondisi().

=== T4-06: Dua Subclass, Behavior Berbeda ===
BarangElektronik : BarangElektronik[kode=E01, nama=Laptop Lenovo, kondisi=BAGUS, merek=Lenovo,
daya=65W]
BarangFurnitur   : BarangFurnitur[kode=F01, nama=Meja Dosen, kondisi=BAGUS, material=Kayu Jati,
warna=Coklat]
→ toString() berbeda: Elektronik tampilkan merek+daya,
  Furnitur tampilkan material+warna

=== T4-07: Regression — PencatatanInventaris + Subclass ===
--- INV-003 ---
=== Hasil Pencatatan ===
Kode : INV-003
Nama Barang : Laptop Lenovo
Kondisi : BAGUS
Lokasi      : Lab Elektronika
Detail Barang   : BarangElektronik[kode=E01, nama=Laptop Lenovo, kondisi=BAGUS, merek=Lenovo, daya=65W]

--- INV-004 ---
=== Hasil Pencatatan ===
Kode : INV-004
Nama Barang : AC Ruangan
Kondisi : BAGUS
Lokasi      : Lab Elektronika
Detail Barang   : BarangElektronik[kode=E02, nama=AC Ruangan, kondisi=BAGUS, merek=Daikin, daya=900W]

--- INV-005 ---
=== Hasil Pencatatan ===
Kode : INV-005
Nama Barang : Meja Dosen
Kondisi : BAGUS
Lokasi      : Ruang Kelas 3A
Detail Barang   : BarangFurnitur[kode=F01, nama=Meja Dosen, kondisi=BAGUS, material=Kayu Jati,
warna=Coklat]

--- INV-006 ---
=== Hasil Pencatatan ===
Kode : INV-006
Nama Barang : Lemari Arsip
Kondisi : HILANG
Lokasi      : Gudang Sarana
Detail Barang   : BarangFurnitur[kode=F02, nama=Lemari Arsip, kondisi=HILANG, material=Besi, warna=Abu-
abu]

[Done] exited with code=0 in 0.847 seconds
```

1. Apa yang membuat Circle layak menjadi class yang lebih umum daripada Cylinder?

Karena Circle bisa berdisi sendiri tanpa Cylinder, tapi Cylinder tidak bisa ada tanpa Circle

2. Berdasarkan Step 5, apa arti praktis extends bagi object Cylinder?

Object Cylinder langsung bisa pakai method milik Circle. Extends membuat Cylinder otomatis membawa semua behavior public dari Circle

3. Mengapa constructor Circle dijalankan ketika Anda membuat object Cylinder?

Karena Cylider Adalah turunan dari Circle dan sebelum bagian Cylinder diinisialisasi bagian Circlenya harus selesai dahulu

4. Mengapa getVolume() menjadi salah setelah getArea() dioverride?

Karena di dalam getVolume() ada pemanggilan getArea(). Setelah getArea() dioverride di Cylinder jadi luas permukaan Cylinder, pemanggilan getArea() di dalam getVolume() otomatis menggunakan versi Cylinder. Jadinya getVolume() menghitung luas permukaan * height, padahal yang benar adalah luas alas * height, dan solusinya pakai super.getArea() yang secara eksplisit minta versi Circle

5. Apa yang secara eksplisit Anda minta ketika menulis super.getArea() atau super.toString()?

Seperti meminta ke Java untuk tidak pakai versi method yang sudah dioverride tapi minta versi asli milik superclass

6. Mengapa Square perlu mengubah setter yang diwarisi dari Rectangle?

Karena setter warisan dari Rectangle hanya mengubah satu dimensi sedangkan untuk Square width harus selalu sama dengan length

7. Apakah setiap dua class yang mirip harus menggunakan inheritance? Jelaskan dari salah satu kasus Anda.

Tidak, syaratnya harus lolos tes ”is-a”. contoh di kasus Inventaris “BarangElektronik is a Barang”. Contoh yang tidak lolos Barang dan Lokasi sama-sama punya field kode dan nama, tapi Barang bukan jenis Lokasi dan Lokasi bukan jenis Barang.

8. Lengkapi dengan bahasa Anda sendiri: "Inheritance bukan sekadar mengurangi duplikasi, tetapi ..."

Inheritance bukan sekadar mengurangi duplikasi tetapi cara memodelkan hubungan nyata. Inheritance membuat kode mengikuti logika nyata bahwa satu hal merupakan versi lebih spesifik dari hal lain